Chess Style Profiler

Adaptive Puzzle Recommendation via Unsupervised Playing Style Fingerprinting

Ishani Bhat

M.S. Data Science, University of Washington
ibhat@uw.edu

Portfolio Project Documentation — Version 2
March 2025

Abstract

This document is the complete technical record of the Chess Style Profiler project — an unsupervised machine learning system that fingerprints chess playing styles from game history and recommends personalized practice puzzles. It covers the full development arc: every design decision, every problem encountered, the reasoning behind each solution, and the underlying ML concepts explained from first principles. It serves simultaneously as a project log, a study guide, and a portfolio artifact.

Contents

1 Project Motivation and Goals **3**
1.1 Origin . . . 3
1.2 The Product Vision . . . 3
1.3 Why This Problem Is Hard . . . 3

2 Data Pipeline **3**
2.1 Data Source: Lichess API . . . 4
2.2 ECO Codes Explained . . . 4
2.3 Problem 1: Famous Players Don't Play Rapid on Lichess . . . 5
2.4 Problem 2: Blitz Games Are Noisy . . . 5
2.5 Problem 3: Sampling Bias — All Players Look the Same . . . 5
2.6 Problem 4: Bot/Outlier Accounts . . . 6
2.7 Final Data Pipeline Summary . . . 6

3 Feature Engineering **6**
3.1 Design Philosophy . . . 6
3.2 Timing Features . . . 6
3.2.1 Panic Score . . . 7
3.2.2 Time Per Move Mean . . . 7
3.3 Move-Level Features (python-chess) . . . 7
3.3.1 Capture Rate vs Sacrifice Rate . . . 8
3.3.2 Check Rate . . . 8
3.3.3 Early Aggression . . . 8
3.3.4 Piece Activity (Development Speed) . . . 8
3.3.5 Average Material Difference . . . 8
3.4 Castle Features: The Key Design Decision . . . 9
3.5 Opening Diversity Features . . . 9
3.5.1 Gambit Rate . . . 9
3.5.2 Sharp Opening Rate . . . 9
3.5.3 Opening Entropy . . . 9
3.5.4 Top Opening Percentage . . . 10
3.5.5 Unique Openings . . . 10
3.6 Outcome Features . . . 10
3.7 Player-Level Aggregation . . . 10

4 Machine Learning Concepts **11**
4.1 Standardization: Why and Which Method . . . 11
4.2 KMeans Clustering . . . 11
4.3 Selecting Optimal k . . . 12
4.4 Gaussian Mixture Models (GMM) . . . 12
4.5 DBSCAN . . . 12
4.6 Hierarchical Clustering . . . 13
4.7 PCA: Dimensionality Reduction Before Clustering . . . 13
4.8 Evaluation Metrics . . . 14

5 Results **14**
5.1 Final Cluster Structure (KMeans, k=4) 14
5.2 Cluster 1 vs Cluster 3: The Key Contrast 14
5.3 Model Comparison Summary 15

6 Full Iteration History **15**

7 Lessons Learned **15**

8 Next Steps **16**
8.1 Immediate 16
8.2 Short Term 16
8.3 Long Term 16

9 Technical Reference **17**
9.1 Repository Structure 17
9.2 Dependencies 17
9.3 Commit History 18

1 Project Motivation and Goals

1.1 Origin

This project was built to address a gap in the chess improvement landscape: existing tools (Lichess puzzles, Chess.com lessons) are generic. They do not adapt to how a specific player actually plays. The goal was to build a system that does exactly that.

Chess was chosen as the domain for three reasons:

- The Lichess platform provides a fully open API with hundreds of millions of real games, including per-move clock data.
- The problem maps directly to what FAANG data science teams work on: user behavior modeling, recommendation systems, and anomaly detection.
- No clean, pre-processed dataset exists — every design decision has to be made from scratch.

1.2 The Product Vision

A web application where a user enters their Lichess username and receives:

1. A **style profile** (aggressive, positional, tactical, defensive).
2. A **weakness analysis** (opening/middlegame/endgame).
3. **Personalized puzzles** from Lichess's 500K+ puzzle database, matched to detected weaknesses.
4. A **progress tracker** measuring improvement over time.

1.3 Why This Problem Is Hard

Core Challenge

There are no ground-truth style labels. Nobody has labeled 100 million chess games as "aggressive" or "positional." This is pure unsupervised learning — the algorithm must discover structure that exists in the data without being told what to look for.

Prior art in this space (Maia Chess, Carnegie Mellon) focuses on move prediction: given a position, what move would a 1500-rated player make? Our approach is fundamentally different — we focus on behavioral fingerprinting across entire game histories, which is more applicable to player improvement.

2 Data Pipeline

2.1 Data Source: Lichess API

All data comes from the Lichess public API (`lichess.org/api`). Lichess is chosen over Chess.com because it is fully open (no authentication required), streams data as newline-delimited JSON (NDJSON), and includes per-move clock data which is essential for time-pressure features.

The core API call fetches games for a single user:

```
url = f"https://lichess.org/api/games/user/{username}"
params = {
    "max":       max_games,
    "clocks":    True,     # seconds remaining after each move
    "opening":   True,     # opening name + ECO classification code
    "moves":     True,     # full move list in SAN notation
    "perfType": "rapid", # time control filter
    "rated":     True,     # rated games only
}
headers = {"Accept": "application/x-ndjson"}
response = requests.get(url, params=params,
                        headers=headers, stream=True)
```

Each game returns a JSON object with the fields in Table 1.

Field	Type	Description
`moves`	string	Space-separated SAN move list
`clocks`	list	Seconds remaining after each move
`opening.eco`	string	ECO classification code (e.g. E94)
`opening.name`	string	Human-readable opening name
`winner`	string	"white", "black", or null (draw)
`status`	string	"resign", "mate", "timeout"
`players`	dict	Usernames and ratings for both sides

Table 1: Lichess API fields used in feature engineering

2.2 ECO Codes Explained

ECO stands for *Encyclopaedia of Chess Openings*. Every opening position is assigned a code:

- **A00–A99**: Flank openings (English, Réti)
- **B00–B99**: Semi-open games (Sicilian, French, Caro-Kann) — generally sharp and aggressive
- **C00–C99**: Open games (1.e4 e5) — tactical
- **D00–D99**: Closed games (Queen's Gambit, Slav) — positional
- **E00–E99**: Indian defenses (King's Indian, Nimzo-Indian) — dynamic

We use ECO codes to classify openings as sharp or positional, and to measure how diverse a player's opening repertoire is.

2.3 Problem 1: Famous Players Don't Play Rapid on Lichess

Issue

When the pipeline was extended to include well-known players (Hikaru Nakamura, Praggnanandhaa, Alireza Firouzja), most returned 0 or 1 games. These players use Chess.com, not Lichess.

Fix

Use the Lichess leaderboard API to programmatically find the most active rapid players on the platform — guaranteed to have games. `https://lichess.org/api/player/top/50/rapid`

2.4 Problem 2: Blitz Games Are Noisy

Issue

Initial data used blitz games (1–5 minutes). In blitz, players move fast due to time pressure, not style. A positional player may blunder into tactical complications with 8 seconds remaining — not because that reflects their style.

Fix

Switch exclusively to **rapid games** (10+0 or 15+10). Players have enough time to execute their actual style.
Interview answer: "I chose rapid over blitz because blitz introduces time-pressure noise that conflates panic behavior with actual stylistic preference."

2.5 Problem 3: Sampling Bias — All Players Look the Same

Issue

Using only the Lichess leaderboard gave us 50 players all in the 2000–2600 rating range. Clustering within a narrow population is like trying to find personality types by sampling only Olympic athletes — everyone looks the same. Silhouette scores stayed around 0.22 with only 2 clusters emerging.

Fix

Switch to fetching players from **Lichess arena tournaments**. Arena tournaments are open to all rating levels, giving us natural diversity from 600-rated beginners to 2600+ grandmasters. We sampled deliberately across rating bands: 600–1200 (beginner), 1200–1600 (casual), 1600–2000 (club), 2000–2400 (advanced), 2400+ (elite).

2.6 Problem 4: Bot/Outlier Accounts

Issue

Player `INDI7` had capture_rate = 0.027, castle_rate = 0.01, and played the same opening in 100% of games. Clearly a bot or test account. Outliers like this distort cluster centroids.

Fix

Filter out players with capture_rate < 0.10. Any human playing real chess captures on at least 10% of their moves. Below this threshold indicates non-human or invalid play.

2.7 Final Data Pipeline Summary

```
# 1. Fetch players from diverse arena tournaments
tournament_players = fetch_tournament_players(tournament_id)

# 2. Sample across rating bands
sampled = sample_by_rating_band(players, bands=[
    (600, 1200), (1200, 1600), (1600, 2000),
    (2000, 2400), (2400, 3000)
], per_band=20)

# 3. Fetch 100 rapid games per player
df = fetch_all_players(sampled, max_games=100)

# 4. Filter: keep players with 50+ rapid games
valid = game_counts[game_counts >= 50].index
df = df[df["username"].isin(valid)]

# Final result: 97 players, 9,496 games
```

3 Feature Engineering

3.1 Design Philosophy

Core Principle

Every feature must answer: "What computable signal from the data corresponds to this behavioral tendency?" Abstract concepts like "aggression" are not features. They must be decomposed into measurable quantities. This is the skill that separates a data scientist from someone who runs models.

3.2 Timing Features

3.2.1 Panic Score

The `clocks` field alternates between white and black: $[c_1^W, c_1^B, c_2^W, c_2^B, \ldots]$

We extract the target player's clocks and compute time per move:

$$t_i = \max(c_i - c_{i+1},\ 0)$$

Panic Score measures whether a player slows down under time pressure:

$$\text{panic_score} = \frac{\bar{t}_{\text{late}}}{\bar{t}_{\text{early}}}$$

where $\bar{t}_{\text{early}}$ = mean time per move over first 30% of moves, and $\bar{t}_{\text{late}}$ = mean over last 30%.

- Score > 1: Slowing down late — panicking under time pressure
- Score < 1: Speeding up — just trying to avoid flagging
- Score $= 1$: Consistent throughout

We use late/early (not early/late) because panic is measured by the *late-game signal*. A high ratio means disproportionately more time late in the game.

3.2.2 Time Per Move Mean

Average seconds spent per move across all games. High = slow, deliberate thinker. Low = fast, intuitive player.

3.3 Move-Level Features (python-chess)

These features require replaying every game move-by-move using the `python-chess` library, which maintains a virtual board state.

```
import chess, chess.pgn, io

pgn_string = f"[Event \"?\"]\n\n{moves_str}"
game = chess.pgn.read_game(io.StringIO(pgn_string))
board = game.board()

for move in game.mainline_moves():
    if board.turn == player_color:
        # Query the board BEFORE pushing the move
        is_capture  = board.is_capture(move)
        is_castling = board.is_castling(move)

        board.push(move)  # update board state

        # Query AFTER pushing for check
        is_check = board.is_check()
```

3.3.1 Capture Rate vs Sacrifice Rate

$$\text{capture_rate} = \frac{\text{captures}}{\text{total moves}} \qquad \text{sacrifice_rate} = \frac{\text{sacrifices}}{\text{total moves}}$$

The distinction between normal captures and sacrifices is critical:

```
PIECE_VALUES = {PAWN:1, KNIGHT:3, BISHOP:3, ROOK:5, QUEEN:9}

moving_piece   = board.piece_at(move.from_square)
captured_piece = board.piece_at(move.to_square)

# Sacrifice = giving up MORE valuable piece for LESS valuable one
if PIECE_VALUES[moving_piece] > PIECE_VALUES[captured_piece]:
    sacrifices += 1
```

Why Separate Sacrifices?

A sacrifice is qualitatively different from a normal capture. It means willingness to give up material for initiative or attack. This is the purest quantifiable signal of aggressive play. A player who sacrifices a knight to take a pawn is making an aggressive, speculative decision — very different from trading equal pieces.

3.3.2 Check Rate

$$\text{check_rate} = \frac{\text{checks given}}{\text{total moves}}$$

Low check rate ≈ positional player (doesn't chase forcing moves). High check rate ≈ attacking player.

Important implementation detail: checks must be detected *after* pushing the move to update board state.

3.3.3 Early Aggression

Counts total captures + checks in the first 10 moves. This captures opening aggression specifically — before the position has stabilized. High early aggression signals gambit and attacking players who try to create chaos before their opponent is developed.

3.3.4 Piece Activity (Development Speed)

Counts non-pawn piece moves in the first 10 moves. High = fast development, active pieces. Low = slow, pawn-heavy opening play.

3.3.5 Average Material Difference

Average material balance (player material minus opponent material) computed at each move:

$$\text{material_diff} = \sum_{\text{player pieces}} v_i - \sum_{\text{opponent pieces}} v_i$$

Positive = tends to win material. Negative = sacrifices material away. Near zero = fair exchanges.

3.4 Castle Features: The Key Design Decision

Issue Encountered

Raw `castle_move` stored the move number when a player castled, with NaN for games where castling never happened. When aggregating across 100 games, two problems arose:

1. NaN values were silently dropped, distorting the mean
2. A player who never castles is making a deliberate choice — that information was being discarded entirely

Fix: Split Into Two Features

- **castle_rate**: What % of games did they castle? Low = risk-taker who skips king safety. Computed as: $\frac{\text{games castled}}{\text{total games}}$
- **avg_castle_move**: When they *do* castle, on what move? Early = safety-first player. Late = willing to delay king safety for attacking chances. Computed only over games where castling occurred.

This preserves both the frequency and timing information, and treats "never castles" as a real behavioral signal rather than missing data.

3.5 Opening Diversity Features

3.5.1 Gambit Rate

Proportion of games with "Gambit" in the opening name. Simple string match. High = gambits are a core part of their repertoire.

3.5.2 Sharp Opening Rate

Proportion of games using aggressive ECO families: B2x–B9x (Sicilian), E6x–E9x (King's Indian), C1x–C4x (Open games).

3.5.3 Opening Entropy

This measures how varied a player's opening repertoire is using information-theoretic entropy:

$$H = -\sum_i p_i \log p_i$$

where p_i is the proportion of games using opening i.

- **High entropy**: Player uses many different openings — flexible, unpredictable, adaptable
- **Low entropy**: Player repeats the same few openings — specialist with a system, predictable

For example: a player who plays the King's Gambit in 80% of games has very low entropy (concentrated distribution). A player who spreads across 60 different openings has high entropy.

3.5.4 Top Opening Percentage

What fraction of games use their single most-played opening? High = specialist. Low = variety player. Inverse signal to entropy.

3.5.5 Unique Openings

Raw count of distinct ECO codes used. Correlated with entropy but captures raw breadth rather than distribution shape.

3.6 Outcome Features

- **win_rate**: Proportion of games won. Correlated with skill level but also with style (aggressive players win or lose decisively, rarely draw).
- **draw_rate**: Proportion of draws. Higher for solid, positional players who accept equal positions.
- **decisive_rate**: Proportion of games ending in a win or loss. $= 1 - \text{draw_rate}$. Note: this is redundant with draw_rate and was later removed from the feature set.
- **avg_game_length**: Mean number of moves per game. Short = decisive, attacking player. Long = positional grinder.

3.7 Player-Level Aggregation

Issue Encountered

Initial clustering used one row per game (100 games per player). After adding 12 features, silhouette dropped from 0.777 to 0.331 and DBSCAN labeled all 96 games as outliers.

Curse of Dimensionality

With n data points in d dimensions, the ratio of maximum to minimum pairwise distance approaches 1 as d grows. Everything becomes equidistant and clustering algorithms fail because they rely on distance. The rule of thumb: you need $\sim 10\times$ more data points than features. With 100 games and 12 features, we needed 1,200 games minimum.

Fix: Switch Unit of Analysis from Game to Player

Aggregate all games per player into a single row of statistics. Style is a property of a player, not of a single game.

- Continuous features: mean + standard deviation
  (mean captures central tendency; std captures consistency)

- Binary/rate features: mean (= rate)
- Castle: split into castle_rate + avg_castle_move

Result: 97 players × 21 features. Far more tractable.

4 Machine Learning Concepts

4.1 Standardization: Why and Which Method

Before clustering, all features are standardized using **StandardScaler**:

$$x' = \frac{x - \mu}{\sigma}$$

This transforms every feature to mean = 0, std = 1.

Why This Is Critical

Without scaling, features with large numerical ranges dominate distance calculations. `avg_game_length` ranges from 32 to 107. `win_rate` ranges from 0 to 1. Without scaling, game length would overwhelm every other signal.

Why StandardScaler over MinMaxScaler? MinMaxScaler compresses everything to [0,1] but is sensitive to outliers — one extreme value stretches the entire range and compresses everything else. StandardScaler is based on mean and standard deviation, making it robust to outliers like our max panic score of 17.7.

4.2 KMeans Clustering

KMeans finds k natural groups by minimizing within-cluster variance.

Algorithm:

1. Initialize k random centroids
2. Assign each point to its nearest centroid (Euclidean distance)
3. Recompute each centroid as the mean of all assigned points
4. Repeat steps 2–3 until assignments stop changing (convergence)

Objective function (inertia):

$$J = \sum_{i=1}^{n} \|x_i - \mu_{c(i)}\|^2$$

where $\mu_{c(i)}$ is the centroid of the cluster assigned to point x_i.

Known failure modes:

- **Local minima**: Bad initialization leads to suboptimal clustering. Fixed with `n_init=20` (run 20 times, keep best).

- **Spherical assumption**: Assumes clusters are roughly round. Fails on elongated or irregular shapes.
- **Must specify** k: Does not discover k automatically.

4.3 Selecting Optimal k

Elbow curve: Plot inertia vs k. Inertia always decreases with more clusters, but the rate of decrease slows. The "elbow" is where adding more clusters gives diminishing returns.

Silhouette score: For each point i:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i),\ b(i))}$$

where $a(i)$ = mean distance to same-cluster points, $b(i)$ = mean distance to nearest other-cluster points. Range: $[-1, 1]$. Higher is better. A score of 0 means the point is on the boundary between clusters.

BIC for GMM: Bayesian Information Criterion penalizes model complexity. Lower is better. Select the k that minimizes BIC.

4.4 Gaussian Mixture Models (GMM)

GMM is like KMeans but softer. Instead of hard assignments, each point gets a *probability* of belonging to each cluster:

$$P(z_k|x_i) \propto \pi_k \cdot \mathcal{N}(x_i|\mu_k, \Sigma_k)$$

Key advantage: Soft assignments. A player gets told "you are 70% Positional Grinder and 30% Tactical Opportunist." This is more realistic than a hard binary assignment — chess styles exist on a spectrum.

In our results, GMM achieved silhouette 0.303 vs KMeans 0.216 — suggesting the clusters are not perfectly spherical, which GMM handles better.

4.5 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) finds clusters as dense regions separated by sparse regions.

Two parameters:

- ε (eps): Maximum distance between two points to be considered neighbors
- `min_samples`: Minimum points to form a dense region

Key advantages over KMeans:

- Discovers k automatically — no need to specify
- Identifies outliers explicitly (label = -1)
- Finds non-spherical clusters

Finding eps automatically (K-Distance Plot): Compute the distance from each point to its k-th nearest neighbor. Sort these distances and plot them. The "elbow" of this curve is the optimal eps. We tried multiple percentiles of the k-distance distribution and selected the one with the best silhouette score.

Final DBSCAN result: eps = 4.639, 2 clusters, 2 outliers, silhouette = 0.337, Davies-Bouldin = 0.933 — our best Davies-Bouldin score overall.

4.6 Hierarchical Clustering

Builds clusters bottom-up: start with each point as its own cluster, then repeatedly merge the two closest clusters until one remains. The result is a tree (dendrogram) that can be cut at any level.

We use **Ward linkage**: merge clusters that minimize the increase in total within-cluster variance. This tends to produce compact, roughly equal-sized clusters.

4.7 PCA: Dimensionality Reduction Before Clustering

Issue Identified

Several features were correlated or redundant:

- `decisive_rate` = 1− `draw_rate` (mathematically derived)
- `castle_rate` and `avg_castle_move` both measure king safety behavior
- `capture_rate` and `sacrifice_rate` are correlated (active capturers also sacrifice more)

Correlated features double-count the same signal and hurt distance-based algorithms.

Fix: Apply PCA Before Clustering

PCA (Principal Component Analysis) finds orthogonal directions of maximum variance and projects the data onto them:

$$\mathbf{X}_{\text{reduced}} = \mathbf{X} \cdot \mathbf{V}_k$$

where $\mathbf{V}_k$ contains the top k eigenvectors of the covariance matrix, selected to retain 90% of variance.

Result: 21 features → 10 PCA components retaining 91.7% variance. The first two components explain 28.9% and 23.7% respectively — likely capturing playing strength (PC1) and playing style/aggressiveness (PC2).

Important: PCA here is used for clustering, not for visualization. When we plot in 2D, that is a separate PCA applied only for display purposes.

4.8 Evaluation Metrics

Silhouette Score measures how well-separated clusters are:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

Higher is better. Our best: DBSCAN at 0.337.

Davies-Bouldin Score measures similarity between each cluster and its nearest neighbor:

$$DB = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right)$$

Lower is better. Our best: DBSCAN at 0.933.

5 Results

5.1 Final Cluster Structure (KMeans, k=4)

With 96 players across all rating levels:

Cluster	Size	Interpretation
0	28	Mid-level players: moderate win rates (0.28–0.57), lower castle rates, shorter games. Inconsistent play.
1	29	Strong, experienced players: high win rates (0.62–0.97), high castle rates (0.83–0.92), solid positional play.
2	5	Aggressive outliers: high capture rates, high sacrifice rates, attacking style.
3	34	Weaker/developing players: low win rates (0.23–0.50), very low castle rates (0.35–0.40), lower capture rates.

Table 2: KMeans cluster interpretation

5.2 Cluster 1 vs Cluster 3: The Key Contrast

The most interpretable comparison:

Feature	Cluster 1 (Strong)	Cluster 3 (Weak)	What It Means
Win rate	0.62–0.97	0.23–0.50	Skill differential
Castle rate	0.83–0.92	0.35–0.40	Structural soundness
Capture rate	0.22–0.24	0.14–0.20	Active play vs passive
Game length	71–94 moves	43–75 moves	Games end faster for weaker players
Opening entropy	3.5–4.0	1.5–2.5	Repertoire diversity

Table 3: Cluster 1 vs Cluster 3 feature comparison

This makes chess sense. Stronger players:

- Castle reliably (king safety is fundamental)
- Capture purposefully (don't leave pieces en prise)
- Have broad opening knowledge (high entropy)
- Grind out longer games (endgame technique)

5.3 Model Comparison Summary

Model	Silhouette	Davies-Bouldin	Notes
KMeans (k=4)	0.204	1.514	4 interpretable clusters
GMM (k=4)	0.182	1.587	Soft assignments (most realistic)
DBSCAN (eps=4.64)	0.337	0.933	Best scores; found 2 outliers
Hierarchical (k=4)	0.159	1.580	Worst performance

Table 4: Final model comparison on 96 players, 21 features + PCA

6 Full Iteration History

Phase	Setup	Best Silhouette	Problem & Fix
1	1 player, 4 features, game-level	0.777	High score but 68/100 in one cluster. Features too few and binary.
2	1 player, 12 features, game-level	0.331	Added features hurt — curse of dimensionality.
3	21 players, 4 features, player-level	0.264	Only 2 clusters. Too few players, leaderboard only.
4	21 players + PCA	0.384 (GMM)	Improvement. Still only 2 clusters. Need more diversity.
5	50 players (leaderboard), 21 features	0.220	Sampling bias. All players from same rating range.
6	97 players (tournaments), 21 features	0.337 (DBSCAN)	Real diversity. 4 meaningful clusters. Best results.

Table 5: Full iteration history

7 Lessons Learned

1. **Data availability shapes problem formulation.** Famous players not being on Lichess forced us toward arena tournament sampling — which is actually more scalable and more diverse.

2. **Feature engineering is where the DS work happens.** Splitting castle_move into castle_rate + avg_castle_move came from domain understanding, not model sophistication. NaN was signal, not noise.

3. **Bad results are diagnostic.** Silhouette dropping from 0.777 to 0.331 after adding features pointed directly to the cause: curse of dimensionality. The right response was to fix the root problem, not try a different model.

4. **The unit of analysis matters more than the algorithm.** Switching from game-level to player-level clustering was a conceptual change, not a technical one. Style is a property of a player, not a single game.

5. **Sampling is as important as modeling.** All 50 leaderboard players gave silhouette 0.220. 97 tournament players gave silhouette 0.337. Same algorithm. The data was the problem, not the model.

6. **Silhouette score alone is not enough.** Phase 1 had silhouette 0.777 but was practically useless (68/100 in one cluster). Always check cluster sizes and interpretability alongside metrics.

8 Next Steps

8.1 Immediate

- Interpret and label all 4 clusters with chess-meaningful names
- Validate clusters by checking if known-style players land in expected clusters
- Add opponent-normalized win rate (remove confounding by opponent rating)

8.2 Short Term

- Integrate Lichess puzzle database (500K+ labeled puzzles with themes: fork, pin, endgame, opening, etc.)
- Build weakness detection: identify which game phase (opening/middlegame/endgame) a player struggles in
- Puzzle-to-weakness matching: given a player's cluster and weaknesses, recommend the right puzzle themes
- Streamlit web app: enter username → get profile + puzzles

8.3 Long Term

- Stockfish evaluation for position quality scoring
- Normalize features by opponent rating differential
- Progress tracking: measure puzzle performance over time
- Publish as Towards Data Science article

9 Technical Reference

9.1 Repository Structure

```
chess-style-profiler/
|-- data/
|   |-- games_raw.csv              # Raw API data (97 players)
|   |-- features_game_level.csv    # Per-game features
|   |-- features_player_level.csv  # Aggregated player profiles
|   |-- clustered_players.csv      # Cluster assignments
|   |-- optimal_k.png              # Elbow + silhouette plots
|   |-- model_comparison.png       # 4-model PCA comparison
|   |-- dbscan_eps.png             # K-distance plot
|-- src/
|   |-- data_loader.py             # Lichess API pipeline
|   |-- features.py                # Feature engineering
|   |-- clustering.py              # All clustering + evaluation
|   |-- recommender.py             # Puzzle matching (planned)
|-- app.py                         # Streamlit app (planned)
|-- requirements.txt
```

9.2 Dependencies

Library	Purpose
`requests`	Lichess API calls
`pandas`	Data manipulation
`numpy`	Numerical computation
`python-chess`	Move parsing, board state replay
`scikit-learn`	Clustering, PCA, scaling, metrics
`scipy.stats`	Entropy calculation
`matplotlib`	Visualization
`tqdm`	Progress bars
`streamlit`	Web app (planned)

9.3 Commit History

Commit	What Changed
initial project structure	Repo, requirements.txt
feat: data loader	API pipeline, Magnus proof of concept
feat: feature engineering	4 game-level features
feat: clustering	KMeans + elbow + silhouette
feat: model comparison	GMM, DBSCAN, Hierarchical
refactor: multi-player rapid	Switch to rapid games
fix: programmatic player fetch	Lichess leaderboard API
feat: move-level features	python-chess integration
feat: player-level aggregation	Castle split, player profiles
feat: PCA before clustering	Dimensionality reduction
feat: opening entropy	Repertoire diversity features
fix: arena tournament sampling	Rating diversity fix
fix: outlier filter + DBSCAN tuning	Bot removal, auto eps
docs: LaTeX project documentation	This document